Baldur: Whole-Proof Generation and Repair with Large Language Models

Автоматизация формальных доказательств с помощью LLM

Тихонов И.Е., Шатских Д.М., Сементин Н.С.

Площадка публикации

ACM/IEEE International Conference on Automated Software Engineering (ASE)

- Одна из ведущих международных конференций в области автоматизации разработки программного обеспечения
- Организуется при поддержке **ACM** и **IEEE**
- Фокус:
  - автоматическая генерация кода
  - тестирование и верификация ПО
  - формальные методы
  - применение искусственного интеллекта в разработке
- Высокий уровень отбора статей (строгое рецензирование)
- Публикует передовые исследования и практические решения

Журнал публикации

Proceedings of the ACM on Programming Languages (PACMPL)

- Международный рецензируемый научный журнал, издаваемый **ACM**
- Ориентирован на исследования в области языков программирования и формальных методов
- Публикации сгруппированы по ведущим конференциям:
  - POPL (Principles of Programming Languages)
  - OOPSLA (Object-Oriented Programming, Systems, Languages & Applications)
  - ICFP (Functional Programming)
- Особенность: статьи проходят строгий peer-review, аналогичный топ-конференциям
- Индексируется в основных научных базах (Scopus, Web of Science)

Рейтинг журнала

Proceedings of the ACM on Programming Languages

United States | Universities and research institutions | Media Ranking

Country

United States

SCIMAGO INSTITUTIONS RANKINGS

SCImago Media Rankings

Subject Area and Category

Computer Science
└ Software

Engineering
└ Safety, Risk, Reliability and Quality

Publisher

Association for Computing Machinery (ACM)

SJR 2024

0.882 Q1

H-Index

54

Publication type

Journals

ISSN

24751421

Coverage

2017-2025

План доклада

1. Часть 1 — Введение и постановка проблемы
2. Часть 2 — Метод Baldur
3. Часть 3 — Эксперименты и результаты

Часть 1 — Введение и постановка проблемы

Почему вопрос надёжности важен

- Большинство систем проверяются тестами.
- Тестирование проверяет **конечный набор** входов, а пространство входов часто огромно/бесконечно.
- Даже хорошо протестированная система может сломаться в редких условиях.

Где это критично

Для систем повышенной ответственности ошибки недопустимы:

- операционные системы;
- компиляторы;
- криптографические библиотеки;
- авиационные и медицинские системы.

Выход — **формальная верификация программ**.

Формальная верификация: что это и зачем

Формальная верификация — математическое доказательство того, что программа удовлетворяет заданным свойствам.

- вместо «проверили на примерах» — «доказали для всех входов»;
- свойства формулируются как логические утверждения;
- результат проверяем машинно.

Примеры свойств:

- сортировка возвращает упорядоченный массив;
- компилятор сохраняет семантику программы.

Цена формальной верификации

- Пример: компилятор **CompCert** (C), корректность доказана формально.
- Известный факт: объём доказательства может быть **существенно больше** объёма кода (в статье приводится пример порядка $\sim 3\times$).

Ключевая проблема: **трудоёмкость написания доказательств**.

Proof assistant и Isabelle/HOL

Proof assistant — среда для формулирования утверждений и написания **машинно-проверяемых** доказательств.

Isabelle/HOL:

- Isabelle — популярный proof assistant;
- HOL — логика высших порядков (удобна для функций, множеств, структур);
- доказательства пишутся как **proof scripts**.

Мини-пример proof script (Isabelle)

```
lemma add_0_right: "x + (0::nat) = x"
by simp
```

И пример с индукцией:

```
lemma add_comm: "(a::nat) + b = b + a"
proof (induct a)
  case 0
  then show ?case by simp
next
  case (Suc a)
  then show ?case by simp
qed
```

Автоматизация доказательств: основные подходы

- **Hammers** — подбирают факты/леммы и пытаются «пробить» доказательство эвристиками. В Isabelle: **Sledgehammer**.
- **Search-based нейропруверы** — предсказывают следующий шаг и ищут доказательство по дереву состояний.

Минусы search-based:

- сложная архитектура (постоянный цикл «модель $\leftrightarrow$ prover»);
- дорогой поиск;
- генерация по шагам ограничивает использование больших LLM.

Переход к Baldur

Авторы предлагают другой подход:

- генерировать **целое** доказательство сразу (whole-proof generation);
- проверять доказательство в Isabelle;
- при ошибке — чинить доказательство (proof repair).

Далее — как устроен Baldur.

Часть 2 — Метод Baldur

Что такое LLM и почему это важно

LLM — большая языковая модель (Transformer), обученная на больших корпусах текста/кода:

- генерирует текст и код;
- умеет «достраивать» последовательности;
- может быть дообучена на специализированных данных (fine-tuning).

В статье используется **Minerva** — модель, предварительно обученная на математических текстах.

Whole-proof generation: идея

Вместо пошагового поиска:

- вход: формулировка теоремы;
- выход: **полный proof script**.

Дальше proof assistant просто проверяет:

- нет ошибок $\Rightarrow$ теорема доказана;
- есть ошибки $\Rightarrow$ пробуем другой сэмпл или включаем repair.

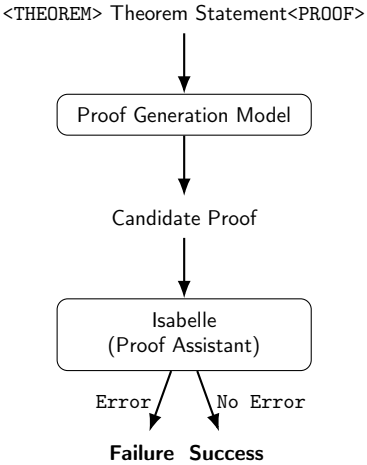


Рис.: An example of using the proof generation model to generate a proof.

Proof repair: как Baldur чинит доказательства

Если Isabelle возвращает ошибку, Baldur формирует вход для repair-модели:

- теорема;
- неверный proof script;
- **сообщение об ошибке** (error message).

Далее модель генерирует исправленный proof script и снова запускается проверка.

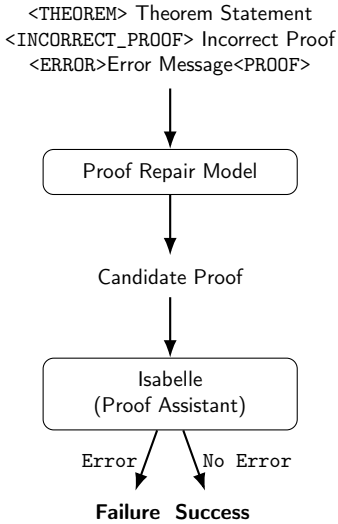


Рис.: An example of using the proof repair model to repair an incorrect proof.

Контекст из файла теории (theory file context)

Дополнительный сигнал: строки **перед теоремой** в том же файле. Они могут содержать:

- определения;
- предыдущие теоремы;
- доказательства;
- комментарии.

Это помогает модели копировать/адаптировать похожие конструкции рядом.

Пример влияния контекста

```
lemma additive_implies_homogenous:
  assumes "forall x y. f (x + y) = f x + f y"
  shows   "f 0 = 0"
by (metis add.left_neutral assms)

lemma fun_sum_commute:
  assumes "f 0 = 0" and "forall x y. f (x + y) = f x + f y"
  shows "f (sum g A) = (\sum a \in A. f (g a))"
```

Почему подход проще, чем step-by-step search

Step-by-step:

- много обращений к модели;
- постоянный цикл «модель $\rightarrow$ prover $\rightarrow$ модель $\rightarrow$ ...»;
- сложнее инфраструктура.

Whole-proof:

- один запрос = много шагов сразу;
- удобно батчить генерацию;
- проверка отдельно на CPU.

Часть 3 — Эксперименты и результаты

Датасет и постановка эксперимента

Оценка проводится на бенчмарке **PISA** для Isabelle/HOL:

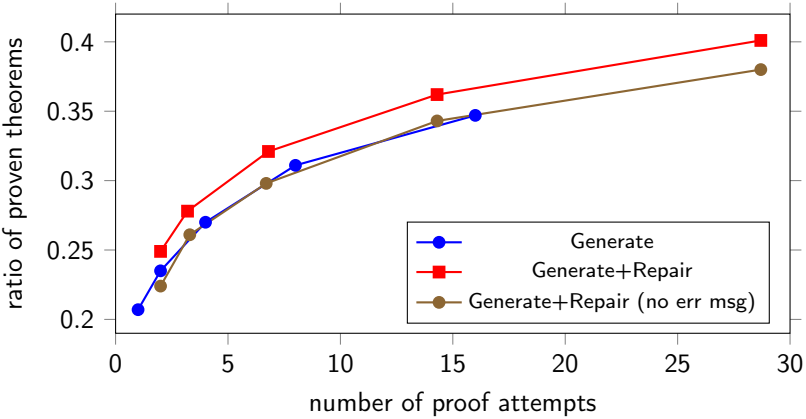
- корпус теорем и доказательств из Isabelle + Archive of Formal Proofs;
- тестовый набор: **6336** теорем.

Меряем долю теорем, доказанных полностью автоматически.

Вопросы исследования

1. Насколько LLM эффективны в генерации **целых** доказательств?
2. Можно ли улучшить результат через **repair** по ошибкам Isabelle?
3. Помогает ли **контекст** из файла теории?
4. Влияет ли **размер** модели?
5. Как Baldur соотносится с state-of-the-art методами (например, Thor)?

Экспериментальные результаты



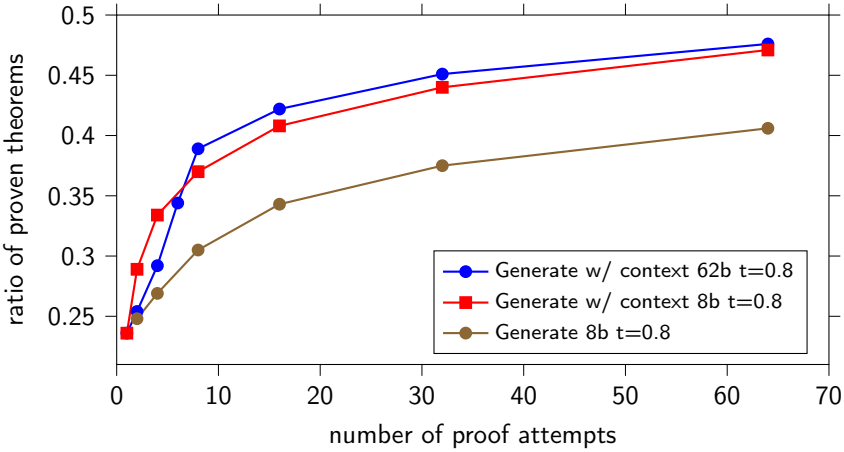
Ratio of theorems proven vs inference cost

Статистика по разным моделям

Model	16 samples	64 samples
Baldur 8b generate	34.8%	40.7%
Baldur 8b generate + repair	36.3%*	—
Baldur 8b w/ context	40.9%	47.5%
Baldur 62b w/ context	42.2%	47.9%
Baldur 8b w/ context $\cup$ Thor	—	65.7%

*Repair uses half the samples and one repair attempt per sample.

Статистика доказательств с использованием контекста



Ratio of theorems proven vs. inference cost for models with different sizes and temperatures.

Комплементарность с Thor

Thor использует комбинацию:

- модели;
- поиска;
- **Sledgehammer** (premise selection/автоматические решатели).

Наблюдение статьи:

- Baldur и Thor решают **частично разные** теоремы;
- объединение результатов даёт около **65.7%** доказанных теорем.

Итоги по эффективности (по тексту статьи)

Ключевые выводы, которые выносим в доклад:

- Baldur доказывает заметную долю теорем автоматически (порядка $\sim 40\%$).
- Repair даёт прирост и важен именно благодаря **error messages** (сообщениям с ошибками).
- Контекст из theory-файла повышает успешность (порядка $\sim 47\%$).
- Более крупные модели работают лучше.

Мини-идея premise selection (почему Thor силён)

```
have "goal"
  by (smt lemma1 lemma2 lemma3)
```

Комментарий:

- успех автоматического решателя зависит от выбора нужных лемм;
- Thor активно использует такие механизмы через Sledgehammer.

Главные выводы

- LLM могут генерировать **полные** формальные доказательства.
- Error messages из proof assistant — полезный сигнал для **proof repair**.
- Whole-proof подход упрощает архитектуру по сравнению с step-by-step search.
- В перспективе: комбинировать подходы (LLM + search/premise selection).

Спасибо! Вопросы

Вопросы?